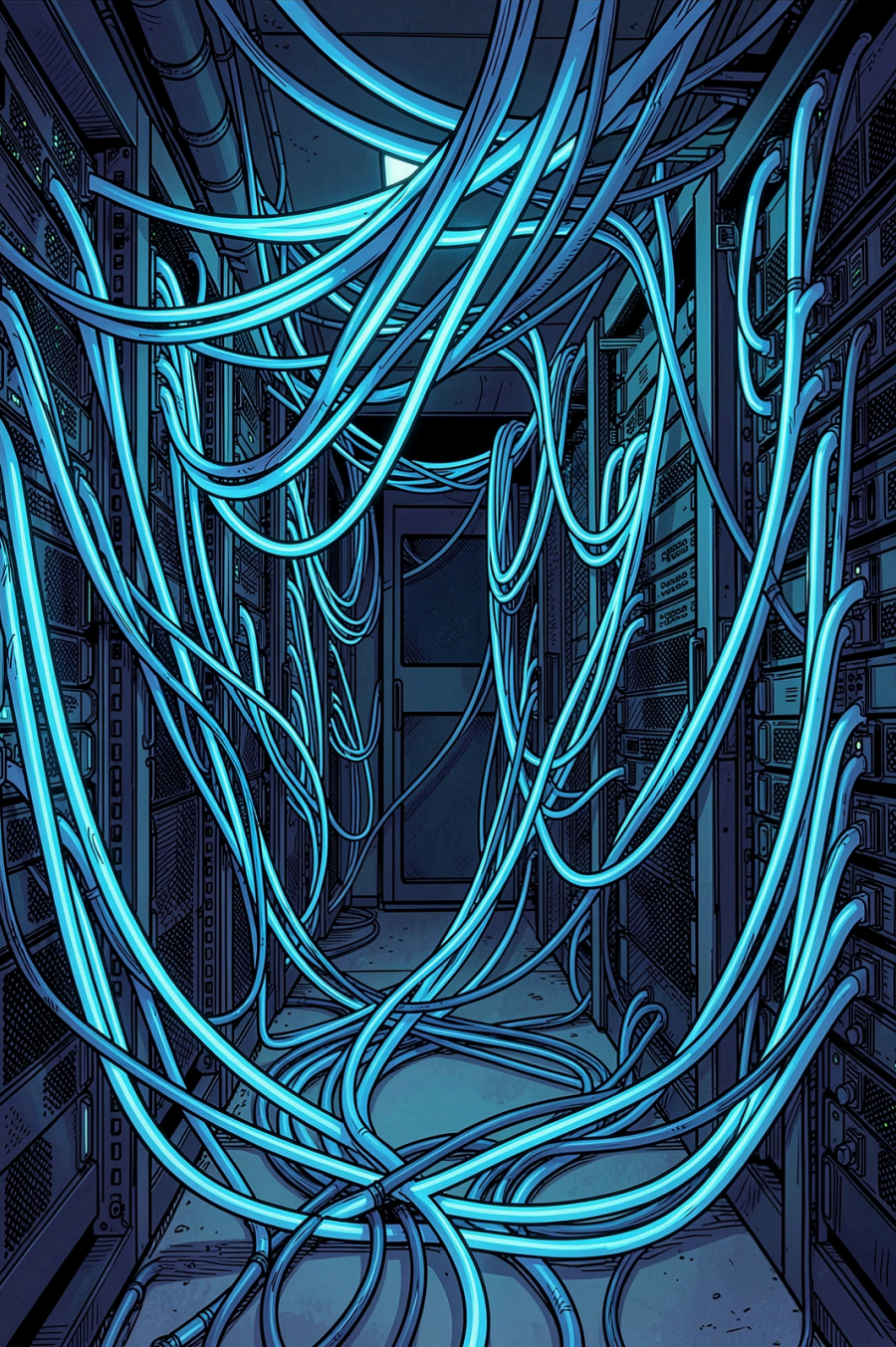




BACKEND ENGINEERING

Building Production-Ready APIs

Small projects return everything. Real systems return exactly what's needed — fast, predictable, and scalable. This session covers the API design patterns that separate toy projects from production-grade backends.

Why Simple CRUD Is Not Enough

Small Projects

- Few records, few users
- Single-table queries
- Minimal load

Real Production Systems

- Thousands of assets, millions of records
- Complex, multi-field queries
- High concurrency and strict latency requirements



Rule: Never return unlimited data. Always paginate.

Pagination: Design the Response

Request

```
GET /assets?page=1&page_size=20
```

Response

```
{  
  "data": [...],  
  "page": 1,  
  "page_size": 20,  
  "total": 145,  
  "total_pages": 8  
}
```

Why This Structure Works

- **Faster responses** — smaller payloads, lower memory
- **Frontend-aware** — client knows exact page count
- **Better UX** — enables reliable navigation controls



Never expose raw database cursors or internal IDs in pagination metadata.

02 – FILTERING

Advanced Filtering

Let clients narrow results to exactly what they need — reducing data transfer and improving usability.

Single Filter

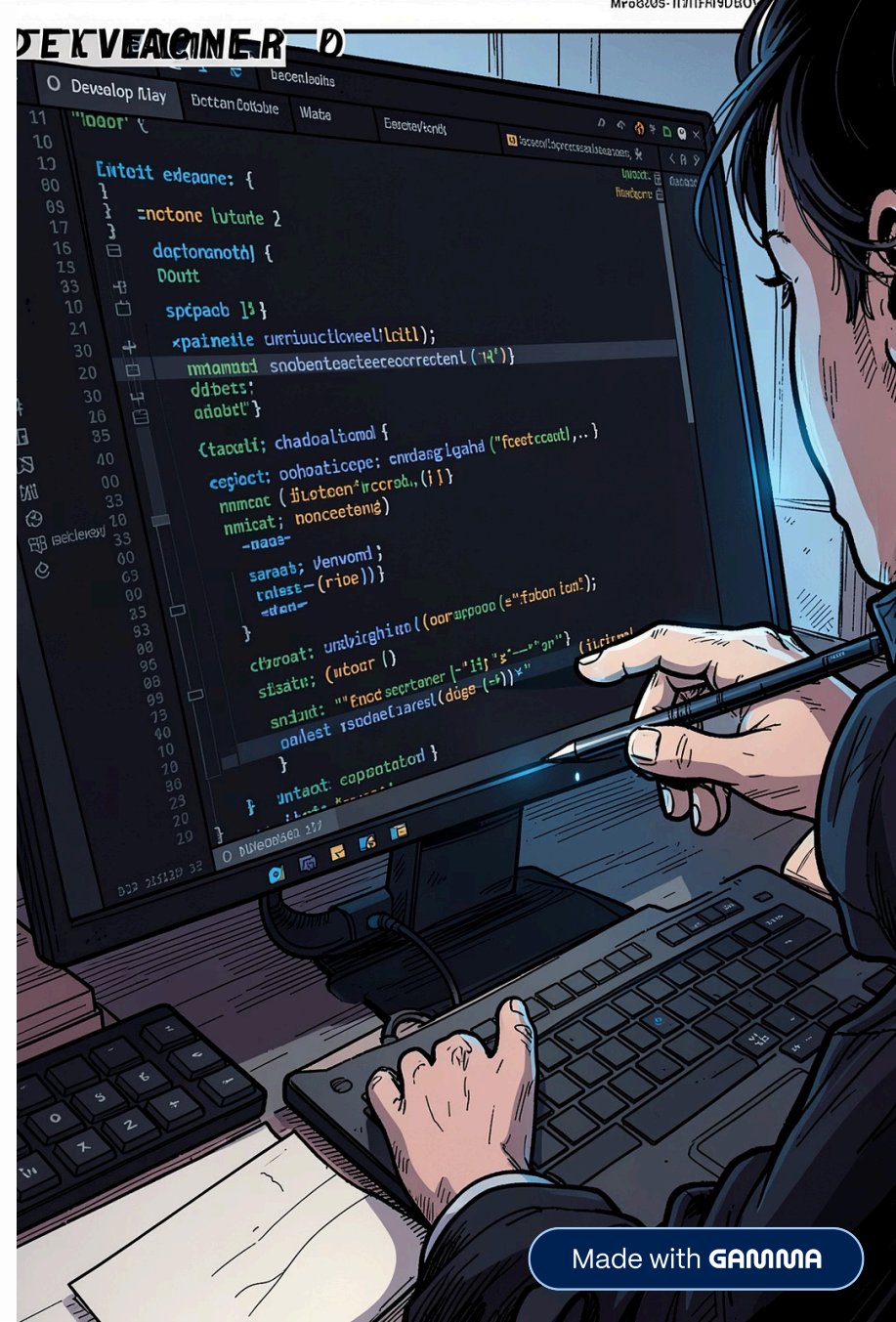
```
GET /assets?type=domain
```

Status Filter

```
GET /assets?status=active
```

Combined Filters

```
GET /assets?type=domain&status=active
```



Search: Handle Ambiguity Gracefully

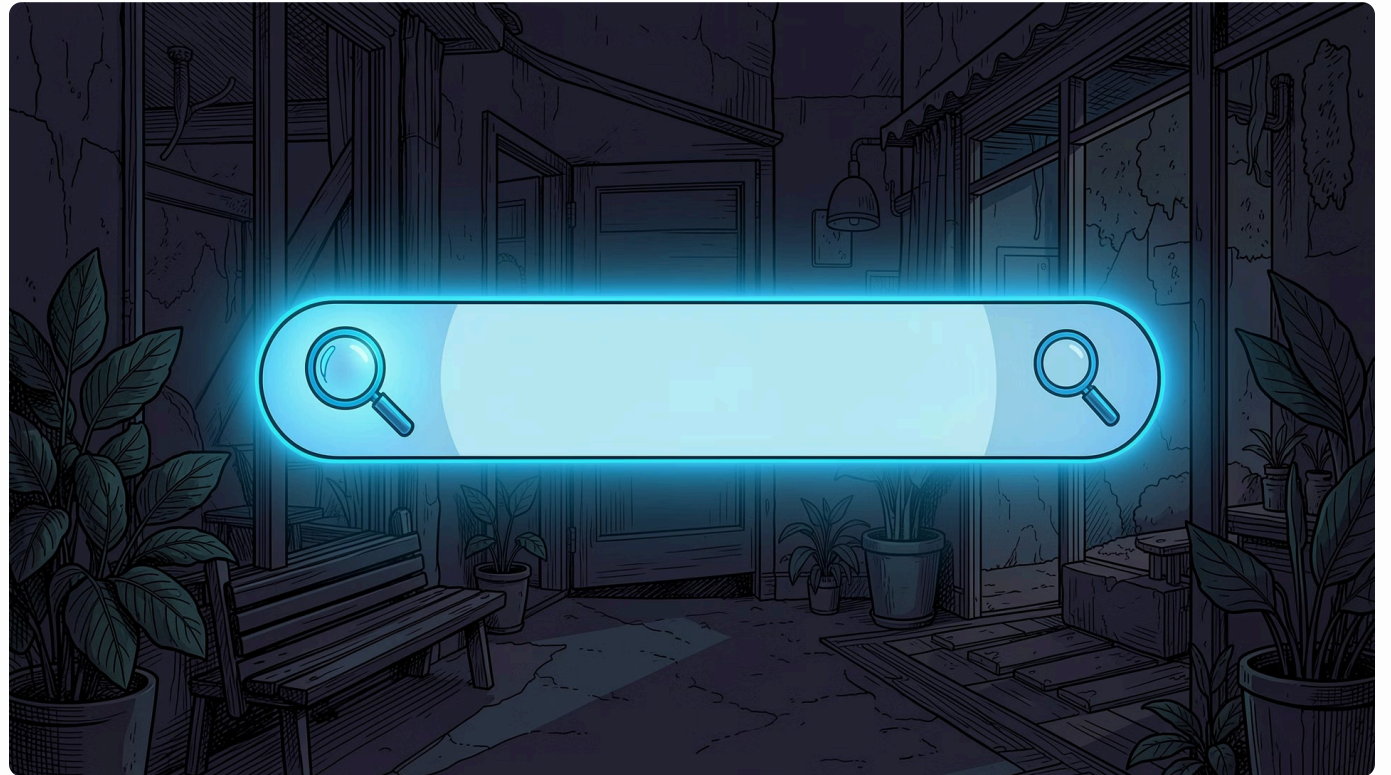
The Problem

Users rarely know exact field values. A `search` parameter enables fuzzy discovery across multiple fields.

```
GET /assets?search=google
```

Matches

- `google.com`
- `api.google.com`
- `mail.google.com`



Sorting: Flexible Data Exploration

Sort Syntax

```
GET /assets?sort=name  
GET /assets?sort=-created_at  
GET /assets?sort=status,name
```

📌 Prefix with `-` for descending order. Multiple fields are comma-separated.

Common Sort Fields

name

status

created_at

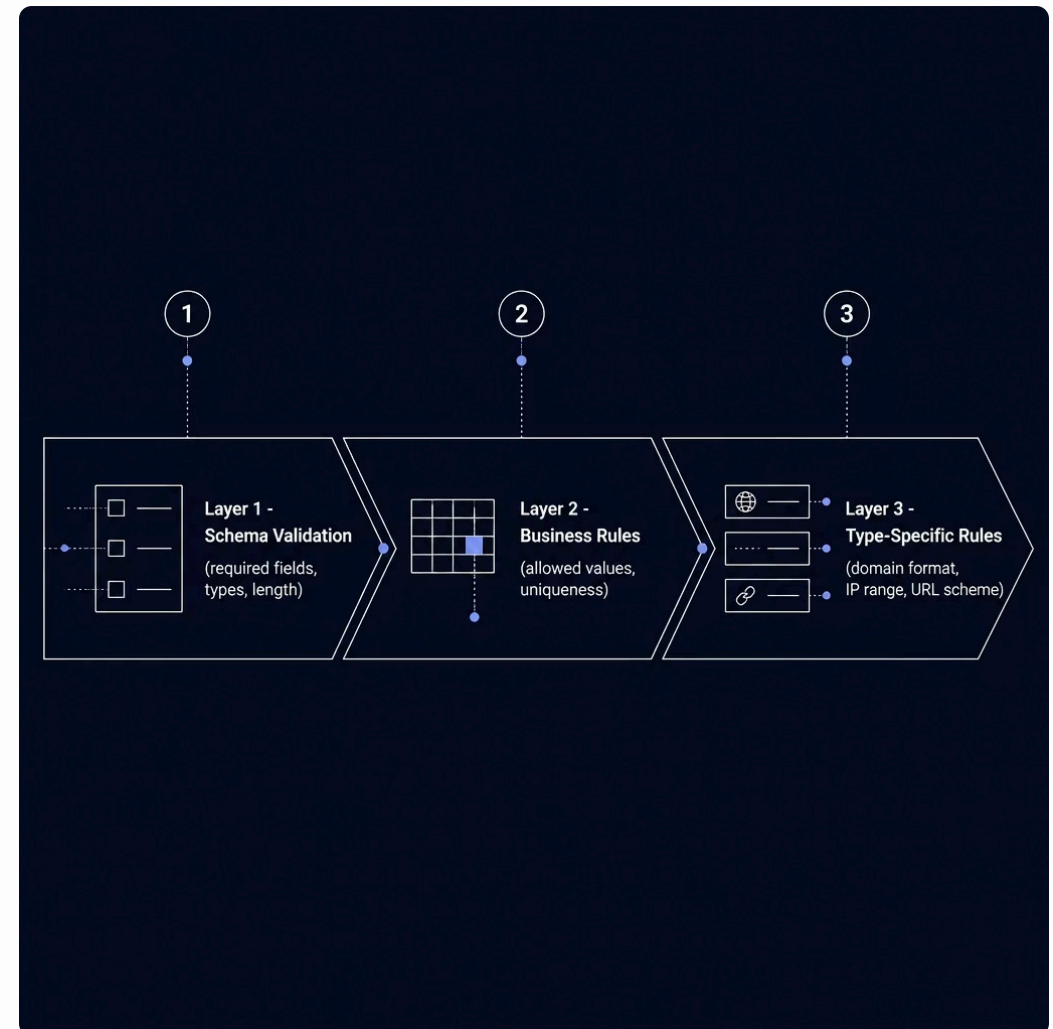
Input Validation: Never Trust the Client

Example Request

```
{  
  "name": "example.com",  
  "type": "domain"  
}
```

Validation Rules

- **name** – required, enforce length limits
- **type** – must be `domain`, `ip`, or `service`
- Reject unknown fields or unexpected types



Type-Specific Validation

Different asset types require different validation logic. Validate as early as possible – ideally before any database interaction.



Domain

example.com

Must match DNS hostname format. No protocols, no paths.



IP Address

192.168.1.1

Validate IPv4 or IPv6 format. Reject reserved ranges if needed.



Service URL

https://api.company.com

Must include scheme (http/https) and valid hostname.

Error Responses Should Be Actionable

❌ Bad Error

```
{  
  "error": "something went wrong"  
}
```

Vague, unhelpful, forces debugging.

✅ Good Error

```
{  
  "error": "invalid asset type",  
  "allowed_values": [  
    "domain",  
    "ip",  
    "service"  
  ]  
}
```

Clear, specific, tells the client exactly how to fix the request.

✅ **Rule:** Errors must be **clear**, **actionable**, and **consistent** across all endpoints.

Production API Checklist

01

Pagination

Never return unlimited data

03

Search

Enable fuzzy discovery

05

Input Validation

Validate early, validate strictly

02

Filtering

Support field-level queries

04

Sorting

Allow flexible ordering

06

Consistent Errors

Clear, actionable, uniform

A good API is not only functional — it is **predictable**, **scalable**, and **easy to use**.